Relay on Codex: running coding-agent plans across ChatGPT usage windows

cube-digital · relay codex-math v0.2 · October 2026 · Codex CLI 0.163.0, read with REA 6.7.0 and measured on a live account · github.com/cube-digital/relay

Relay turns a written plan into parallel work for the cheapest model that can do each task and keeps the plan running across usage windows. This page is the Codex counterpart of the relay whitepaper: what the Codex harness meters and exposes (recovered from the binary, checked on a live Business account), what that changes in relay's mechanism, the model behind the Codex forecast, and what the first measurements say. Facts and evidence: codex-math/config/codex-harness.json, codex-math/evidence/; model code: codex-math/lib/.

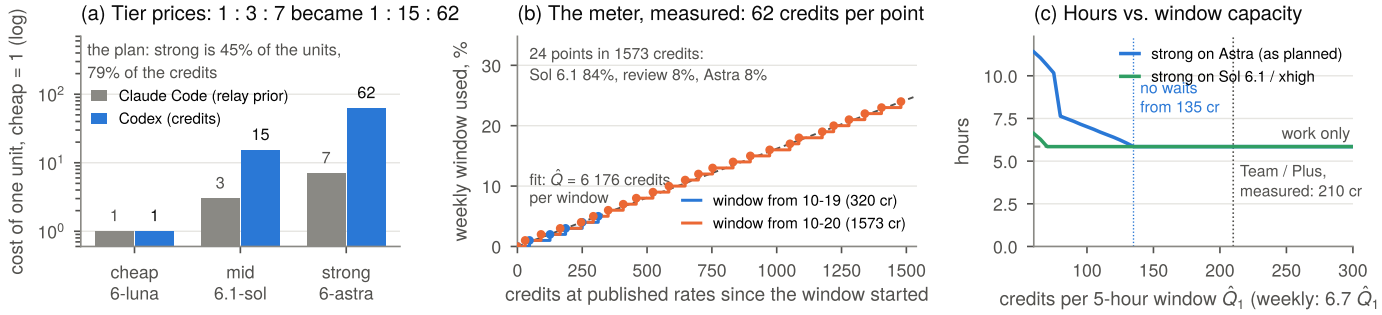


Figure 1. (a) The relay whitepaper's 30-task plan (7 lanes, 3 waves, 108.4 units, 4.7 expected fix rounds) on Codex: credits per unit by tier, Luna : Sol 6.1 : Astra = 1 : 15 : 62 at the measured token mix and meter weights, where relay's Claude prior was 1 : 3 : 7; the plan's strong tasks are 45% of its units and 79% of its 131 credits. (b) The meter, measured: a Business account's weekly window against the credits its threads spent at the published rates, two windows, 29 integer steps, $\hat{Q} = 6\,176$ credits per window, i.e. 62 credits per point. (c) Hours for the plan against the 5-hour capacity $\hat{Q}_1$: at the Team prior of 210 credits it runs its 5 h 51 min with no pause (none above 135 credits); with the strong tier on Sol 6.1/xhigh it costs 66 credits. Computed by data.mjs with the model code.

1 What the harness meters

Read from @openai/codex 0.163.0 with REA: process captures of the binary (help, exec --json, the TUI, the app-server over stdio), its protocol schema and model catalog, the strings of its .rodata, two defaults decompiled from its code; then checked on a live Business account (self_serve_business_prolite): 114 rollouts mined, nine priced runs, a spawn-limit run. The credit rates are OpenAI's published ones.

- **Windows are plan-shaped, not fixed.** A reading is a RateLimitSnapshot: primary and secondary windows with usedPercent, resetsAt and windowDurationMins, plus planType, credits and a per-pool view keyed by limit_id. Team meters a 5-hour and a weekly window; Pro and the Business plan measured here only the weekly one. usedPercent is an integer; an unused window's resetsAt slides with the clock until the first metered use, and a mid-window reset shows as a changed resetsAt. Readings come from account/rateLimits/read (and /updated) on the app-server and from the token_count event in each rollout file; API-key sessions get none.
- **Credits are the unit, with weights.** Published rates per million tokens, input / cached / output: Luna 2.5 / 0.25 / 12.5, Sol 6.1 50 / 2.5 / 250, Astra 250 / 25 / 1250; Fast meters 2.5× the included usage (2× purchased credits) for a 1.5–2× speedup; Ultrafast 8×. The weekly window moved linearly in these credits (Fig. 1b, residual 0.6 points) with one correction: Astra work counts about half its published rate (0.43–0.57, five steps). The backend's per-thread estimate (account/usage/read {threadId}) is null on this plan, so the published rates are the only per-thread price.
- **Tokens are attributable.** Every subagent is its own thread: state_5.sqlite holds its model, effort and token total, thread_spawn_edges its parent; exec --json ends with turn.completed.usage. Real sessions are 97% cached input by tokens. The usable context is 95% of the window and auto-compaction triggers at 90% (decompiled): 258 400 and 244 800 tokens on the 272 k window of Astra and Luna, 475 000 and 450 000 on Sol 6.1 (500 k); tool output is cut at 10 000 tokens; compaction is hookable (PreCompact, PostCompact).
- **Levers relay did not have.** Spawns take model, effort and speed; [agents] sets the subagent defaults and max_concurrent_threads_per_session the children (default 3: the compiled limit is 4 threads with the root, and the 4th spawn fails with *agent thread limit reached*); a PreToolUse hook can rewrite a spawn_agent call through updatedInput (the Codex form of relay's mod); reset credits restore every window at once; a thread *goal* with a token budget keeps an idle thread going until budget_limited or usage_limited (relay's marathon, built in).

2 What changes in the mechanism

Tiers keep their roles, but the ladder is 1 : 15 : 62, so the cheap tier is nearly free: Luna costs 1/15 of Sol per unit, still 1/5 at three times the turns; Astra is spent only where the plan says strong. Lanes run at most three abreast unless [agents] raises the limit. Budget gates degrade in price order: speed to standard, effort to medium, then strong to mid; the architect is exempt at tight; a weekly window is gated by the weekly lines wherever the account reports it. A reset credit is spent when the next pause exceeds an hour. The ledger stamp carries the account's windows and the credits spent: [primary 23.4% resets 16:30 · secondary 41% · credits 12.5].

3 The model

Units. As in relay, task t weighs $w_t = T(\text{tier}_t)R(\text{risk}_t)$ with $T = (1,3,7)$, $R = (0.8,1,1.4)$, read now as relative token volume. On tier τ a unit processes ν_τ tokens (prior 70 k) in the measured shares $\alpha = (0.03, 0.965, 0.005)$ of uncached input, cached input and output; with the model's rates p_τ, its meter weight ω_τ (Astra 0.5, else 1), the speed multiplier s_τ and the effort multiplier e_τ on output, a unit costs

$$c_\tau = \omega_\tau s_\tau \nu_\tau \left(\alpha_{\text{in}} p_\tau^{\text{in}} + \alpha_{\text{c}} p_\tau^{\text{c}} + \alpha_{\text{out}} e_\tau p_\tau^{\text{out}}\right)/10^6$$

credits: 0.027, 0.41 and 1.65 at the priors. Nine runs measured 0.018–0.026 and 0.30–0.49 for the first two and 0.74 metered (1.48 published) for strong: a conservative prior. Lane l costs $C_l = \sum_{t\in l} w_t c_{\tau(t)}$.

Fix rounds. Unchanged: lane l needs a fix round with probability $p_l \in \{0.2, 0.5, 0.8\}$ re-working $\kappa = 0.35$ of it, so $\tilde{C}_l = C_l(1+\kappa p_l)$, $\mathrm{E}[C] = \sum_l \tilde{C}_l = 131$ and $\mathrm{Var}[C] = \kappa^2 \sum_l C_l^2 p_l(1-p_l)$, SD = 8.4; the exact distribution over the 2^7 outcomes agrees.

Windows. Window j holds Q_j credits, so the plan uses $U_j = \mathrm{E}[C]/Q_j$ of it: 62% of a Team 5-hour window ($Q_1 = 210$), 9% of its week ($Q_2 = 1\,400$), 2% of a Business week ($Q = 6\,176$). The walk runs a wave while every window is below its stop line (85% primary, 95% secondary), spends credits and time linearly, pauses at a line until that window resets, rolls windows over on schedule, and spends a reset credit instead of a long pause when one is available. Wave time is $m\,M_k$ with M_k the makespan of the lanes' expected units on three slots; Fast divides a lane's minutes by its speedup and multiplies its credits by 2.5, which prices the trade per wave (wave 1 saves 32 min for 10 credits, wave 2 saves 93 min for 102).

Calibration has two layers that never share a parameter. Capacity: between two readings with no reset, the credits spent and the points the window moved give $\hat{Q}_j = 100 \sum \Delta C / \sum \Delta u_j$, a ratio estimator over all such intervals, independent of the plan; readings of concurrent threads are pooled on the account's credit timeline against each window's high-water mark, which made Fig. 1b from a machine's rollouts before relay ran at all. Price per unit: every lane thread's credits are known, so $\hat{c}_\tau = \sum C / \sum w$ over the threads on tier τ, at one model, effort and speed; with only wave totals, non-negative least squares of ΔC_k on the units per tier. Time: $\hat{m} = \sum \Delta t_k / \sum M_k$ as before. Each estimate replaces its prior from the first sample.

4 What is not claimed

- Measured once, on one machine: $\hat{Q}$ for the Business week (29 steps, two windows, days within 10%), Team's 5-hour and weekly windows (two windows each: 212/277 and 1 300/1 640 credits; usage on other devices is invisible to a rollout and biases $\hat{Q}$ low), Pro's week (four steps, 12 600–18 100); Plus is assumed equal to Team. ν_τ, e_τ, T, R, p_l and κ are still priors: the scratch runs were too small to show effort in the output, and a relay unit has not been timed on Codex.
- The meter weights are relative to Sol 6.1 and fitted where the account ran other models; Luna's is unmeasurable there (5 credits) and left at 1. Rates and catalog change without a binary change (Sol 6.1's window did); a mid-thread model, effort or speed change re-reads the context at the uncached rate.
- Lanes are independent and front-load usage less than real runs do; the walk ignores spend controls and pools other than the primary limit_id. Savings are not measured yet: next, ten calibrated waves, then an A/B of the same plans on Codex plain and with relay.

Evidence: codex-math/evidence/ (REA captures, schemas, catalog, SQLite DDL, .rodata strings; live/; decompile/; RESULTS.md); docs cited in codex-harness.json. Rebuild: docs/whitepaper/README.md.